Рекуррентные нейронные сети (часть 2)

Рекуррентные нейронные сети (часть 2) Лекция 12

Седых И.А.

Проблема переобучения в RNN/LSTM

Рекуррентные сети, особенно LSTM с большим количеством параметров, склонны к переобучению, особенно при работе с небольшими или зашумленными наборами данных. Для решения этой проблемы применяется регуляризация, которая накладывает ограничения на процесс обучения, делая модель более устойчивой и простой.

Переобучение в **RNN/LSTM** проявляется в том, что модель начинает «подстраиваться» под случайный шум и нерепрезентативные особенности обучающей последовательности, а не под её истинные закономерности.

В контексте последовательных данных это особенно критично, так как модель может запомнить **не общие правила языка или динамики процесса**, а **конкретную последовательность слов или значений**.

Стандартные методы регуляризации, хорошо зарекомендовавшие себя в свёрточных и полносвязных сетях, не могут быть напрямую перенесены на рекуррентные сети из-за наличия рекуррентных связей, которые многократно усиливают любой вносимый шум.

Dropout в рекуррентных сетях

Dropout - один из самых эффективных методов регуляризации для сетей прямого распространения. Его идея заключается в случайном «выключении» (обнулении) части нейронов на каждом шаге обучения, что заставляет сеть распределять знания более равномерно.

Войцех Заремба, Илья Суцкевер и Ориол Виньялс в своей работе «Recurrent Neural Network Regularization» в 2014 г. показали, что обычное применение dropout к рекуррентным связям, т.е. к соединениям, передающим состояние h_{t-1} в h_t, приводит к катастрофическим последствиям.

Проблема в том, что случайный шум, вносимый dropout на каждом временном шаге, **многократно усиливается рекуррентной петлей**. Это приводит к экспоненциальному «забыванию» полезной информации и делает невозможным обучение долгосрочным зависимостям, ради которых и создавались LSTM.

Заремба и соавторы предложили **применять dropout только к нерекуррентным связям.**

Такой подход позволяет сети извлечь выгоду из регуляризации dropout, не разрушая при этом её способность хранить информацию в клеточном состоянии C_t на протяжении многих временных шагов. Авторы показали, что этот метод **значительно снижает переобучение** в задачах языкового моделирования, распознавания речи, машинного перевода и генерации описаний изображений.

Корректное применение dropout в LSTM

Формула с корректным применением dropout:

$$\begin{pmatrix} i_t \\ f_t \\ o_t \\ \tilde{C}_t \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} \left(W \cdot \begin{pmatrix} \mathbf{D}(h_t^{l-1}) \\ h_{t-1}^{l} \end{pmatrix} \right)$$

Левая часть - выходные векторы (гейты), которые вычисляются одновременно.

После равно записан вектор функций активации.

W - матрица весов, представляет собой **конкатенацию (объединение)** всех четырех матриц весов LSTM.

Входной вектор с Dropout: $\begin{pmatrix} \mathbf{D}(h_t^{l-1}) \\ h_{t-1}^{l} \end{pmatrix}$

Это входной вектор-столбец, на который умножается матрица *W*. Он состоит из двух частей:

- **Верхняя часть:** $\mathbf{D}(h_t^{l-1})$. Здесь $\mathbf{D}$ — это оператор **Dropout**. h_t^{l-1} — это выход с **предыдущего слоя** $l-1$ в текущий момент времени t (в случае первого слоя это просто входной вектор x_t).
- **Нижняя часть:** h_{t-1}^{l}. Это скрытое состояние **текущего слоя** l с предыдущего временного шага $t-1$. **Обратите внимание:** к этой части оператор Dropout **НЕ** применяется.

Таким образом:

- **Dropout накладывается на "внешний" сигнал** h_t^{l-1}, делая вход зашумленным и заставляя сеть не полагаться на конкретные признаки. Это регуляризует входные и выходные (межслойные) связи.
- **Рекуррентная связь** h_{t-1}^{l} **остаётся нетронутой.** Благодаря этому информация в клеточном состоянии C_t может беспрепятственно течь сквозь время, не разрушаясь случайным шумом от Dropout.

Recurrent Dropout для RNN

Дальнейшим развитием идей регуляризации RNN стала работа Ярина Гала и Зубина Гахрамани в 2016 г. «A Theoretically Grounded Application of Dropout in Recurrent Neural Networks» («Теоретически обоснованное применение дропаута в рекуррентных нейронных сетях»).

Они предложили **вариационный (рекуррентный) dropout**, ключевая идея которого заключается в том, чтобы использовать **одну и ту же маску dropout на всех временных шагах** для рекуррентных связей.

В отличие от «наивного» dropout, где маска генерируется заново на каждом шаге, здесь она фиксируется для всей развернутой последовательности.

Этот метод состоит в том, что dropout накладывается не на активации, а на **веса рекуррентных связей**, и эта «маска весов» остается постоянной во времени.

Такой подход позволяет регуляризовать и рекуррентные связи, не вызывая экспоненциального затухания информации, и часто превосходит по качеству «нерекуррентный» dropout .

Другие методы регуляризации RNN - 1

1. L1 и L2-регуляризация (Weight Decay) - это классический метод, который добавляет штраф к функции потерь за большие значения весов:

- **L2-регуляризация:** $L_{total} = L_{original} + \lambda \sum \|W\|_2^2$. Штрафует большие значения весов, заставляя их оставаться близкими к нулю.
- **L1-регуляризация:** $L_{total} = L_{original} + \lambda \sum \|W\|_1$. Штрафует сумму абсолютных значений весов, что часто приводит к разреженным (sparse) матрицам весов.

Этот метод универсален и применяется ко всем весам сети, включая рекуррентные.

2. Нормализация по слою (послойная нормализация, Layer Normalization) предложена Джимми Ба и др. в 2016 г. и является мощной альтернативой batch normalization для RNN.

Batch normalization плохо работает с рекуррентными сетями из-за зависимости от статистики по батчу, которая некорректна для развернутых последовательностей. Layer normalization, напротив, нормализует активации **внутри одного обучающего примера** (по признакам), что делает её независимой от размера батча и длины последовательности. Она стабилизирует обучение и позволяет использовать более высокие learning rates, ускоряя сходимость.

Другие методы регуляризации RNN - 2

3. Градиентное клиппирование (Gradient Clipping) - этот метод в первую очередь решает проблему **взрыва градиента** и также является важным стабилизатором обучения, который косвенно предотвращает «вылет» модели в область хаотических весов, что можно рассматривать как форму регуляризации. Суть метода в ограничении нормы вектора градиента некоторым пороговым значением (например, 1, 5 или 10):

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1, \frac{\text{threshold}}{\|\mathbf{g}\|}\right)$$

где threshold (порог) - гиперпараметр, задаваемый пользователем, определяет максимально допустимую норму градиента.

В рекуррентных сетях из-за многократного умножения на матрицы весов при разворачивании во времени градиенты могут экспоненциально расти. Один «плохой» батч с взрывным градиентом может необратимо испортить веса, отправив их в область, откуда сеть уже не восстановится.

Градиентное клиппирование - это простой и эффективный «предохранитель», который режет пиковые значения, не позволяя одному выбросу разрушить весь процесс обучения.

Другие методы регуляризации RNN - 3

4. Ранняя остановка (Early Stopping) - самый простой и часто используемый метод. Обучение останавливается в тот момент, когда ошибка на валидационной выборке перестает уменьшаться. Это предотвращает «зазубривание» обучающих данных и сохраняет модель в точке её наилучшей обобщающей способности.

5-6. Регуляризация активаций (Activation Regularization, AR) и Временна́я регуляризация активаций (Temporal Activation Regularization, TAR) были предложены в работе Стивена Мерити (Stephen Merity), Брайана Макканна (Bryan McCann) и Ричарда Сочера (Richard Socher) из Salesforce Research в августе 2017 г. под названием «Revisiting Activation Regularization for Language RNNs» («Переосмысление регуляризации активаций для языковых рекуррентных сетей»).

Другие методы регуляризации RNN - 4

Activation Regularization (AR) - в то время как традиционная L2-регуляризация (Weight Decay) штрафует большие значения весов сети, AR штрафует большие значения самих активаций (выходов скрытого состояния) RNN. Это заставляет модель генерировать более сглаженные, умеренные внутренние представления, предотвращая чрезмерную самоуверенность на отдельных признаках и делая модель более устойчивой к шуму во входных данных: $L_{AR} = \alpha \cdot L_2(m \odot h_t) = \alpha \cdot \|m \odot h_t\|_2^2$

- h_t — выход RNN (скрытое состояние) на временном шаге t,
- m — маска dropout, применяемая к выходу h_t,
- $\| \cdot \|_2$ — L2-норма (сумма квадратов элементов вектора),
- α — коэффициент масштабирования (гиперпараметр)

Маска dropout m - это вектор той же размерности, что и скрытое состояние h_t, состоящий из случайных значений {0,1}.

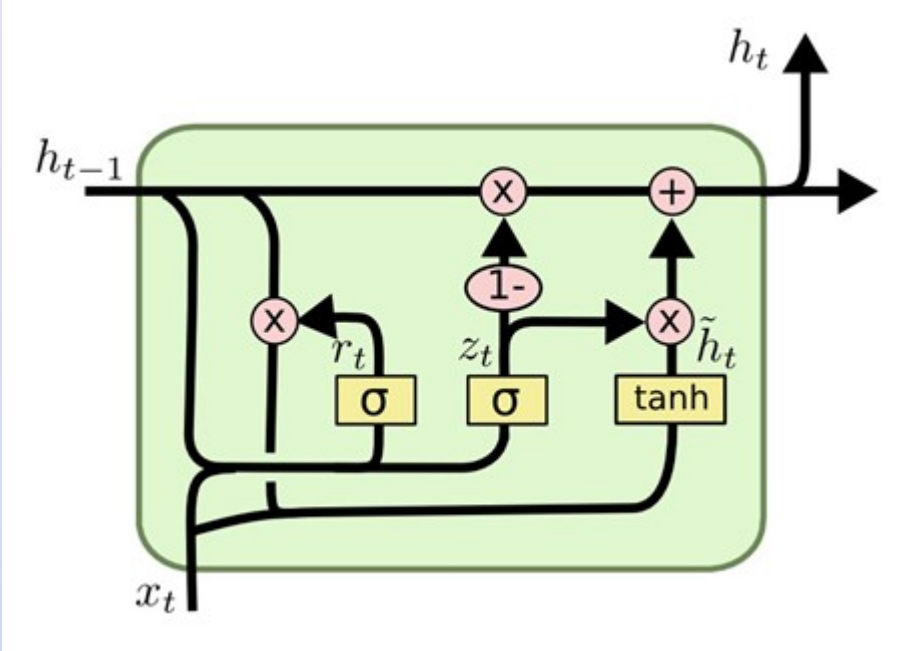
Temporal Activation Regularization (TAR) реализует принцип «медленности», который гласит, что хорошие скрытые представления последовательности должны меняться плавно, а не скачкообразно от слова к слову. TAR штрафует модель за **резкие изменения** скрытого состояния между соседними временными шагами. Это поощряет модель сохранять контекст более стабильным и меньше реагировать на случайные выбросы: $L_{TAR} = \beta \cdot L_2(h_t - h_{t+1}) = \beta \cdot \|h_t - h_{t+1}\|_2^2$

- h_t и h_{t+1} — выходы RNN на соседних временных шагах,
- β — коэффициент масштабирования (гиперпараметр)

AR применяется к активациям **после** dropout, **TAR - до** dropout.

GRU (2014)

GRU (управляемый рекуррентный блок, Gated Recurrent Unit) - упрощенная версия LSTM, предложенная в 2014 году Кюнгхюном Чо (Kyunghyun Cho) и его коллегами. У неё **два гейта**: reset gate r_t (какую часть прошлого забыть) и update gate z_t (аналог объединения forget и input gate). Нет отдельного cell state, есть только hidden state.



$$z_t = \sigma\left(W_i[h_{t-1}, x_t] + b_z\right)$$

$$r_t = \sigma\left(W_r[h_{t-1}, x_t] + b_r\right)$$

$$\tilde{h}_t = \tanh(W[r_t h_{t-1}, x_t])$$

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t$$

GRU обучается быстрее и требует меньше памяти, чем LSTM, но LSTM может быть мощнее на некоторых задачах.

Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, Yoshua Bengio Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation, 2014.

Практические рекомендации по выбору GRU/LSTM

Когда выбрать GRU:

- Данных мало (риск переобучения у LSTM выше)
- Важна скорость обучения
- Последовательности не очень длинные (до 50-100 шагов)
- Нужно быстро прототипировать

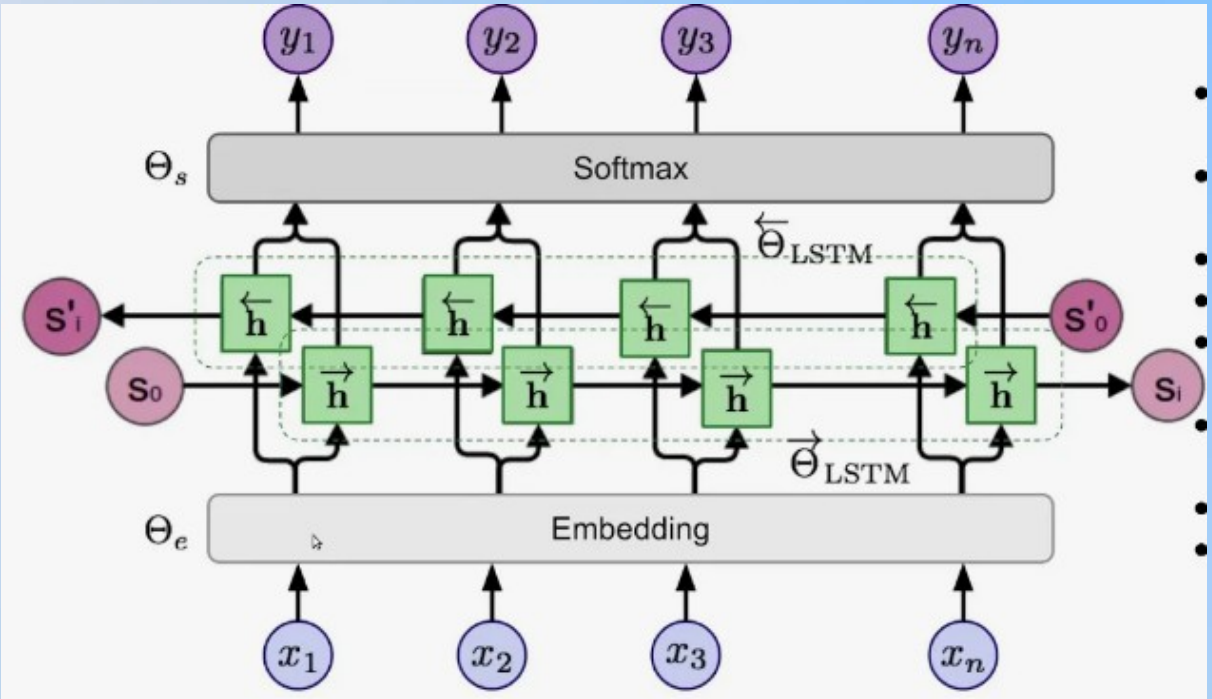
Когда выбрать LSTM:

- Очень длинные последовательности (сотни-тысячи шагов)
- Нужно максимальное качество
- Много данных (переобучение не проблема)
- Важна интерпретируемость (явное клеточное состояние)

GRU имеет на 25% меньше параметров, что ускоряет обучение и снижает риск переобучения. Начинать рекомендуется с GRU. Если качество неудовлетворительное, то нужно перейти на LSTM.

Двунаправленная LSTM (2005) - 1

Стандартная LSTM обрабатывает последовательность в одном направлении. **Двунаправленная LSTM (Bidirectional LSTM, BiLSTM)**, предложенная Алексом Грейвсом (Alex Graves) и Юргеном Шмидхубером (Jürgen Schmidhuber) в 2005 г. в работе «Framewise phoneme classification with bidirectional LSTM and other neural network architectures» («Пофреймовая классификация фонем с использованием двунаправленных LSTM и других архитектур нейронных сетей»), запускает **две независимые LSTM**: прямую (LSTM→) и обратную (←LSTM). На каждом шаге их скрытые состояния конкатенируются, обеспечивая доступ к полному контексту.



Двунаправленная LSTM (2005) - 2

Двунаправленная LSTM - это расширение стандартной архитектуры LSTM, предназначенное для задач, в которых **будущий контекст** (элементы справа от текущего) так же важен, как и прошлый.

Архитектура BiLSTM:

1. Прямой слой (LSTM→) обрабатывает исходную последовательность в естественном порядке $x_1, x_2, \ldots, x_T$. На каждом шаге t он генерирует скрытое состояние, основанное на прошлом контексте: $\overrightarrow{h}_t = \mathrm{LSTM}(x_t, \overrightarrow{h}_{t-1}, \overrightarrow{C}_{t-1})$

2. Обратный слой (←LSTM) обрабатывает ту же последовательность, но в обратном порядке: $x_T, x_{T-1}, \ldots, x_1$. На каждом шаге t он генерирует скрытое состояние, основанное на будущем контексте (относительно исходного порядка): $\overleftarrow{h}_t = \mathrm{LSTM}(x_t, \overleftarrow{h}_{t+1}, \overleftarrow{C}_{t+1})$

3. Объединение (конкатенация) - итоговое скрытое состояние на шаге t формируется путем **конкатенации** (склеивания) векторов прямого и обратного проходов:

$$h_t^{bi} = [\overrightarrow{h}_t; \overleftarrow{h}_t]$$

Размерность итогового вектора равна $2 \cdot d_h$, где d_h - размерность скрытого состояния одного LSTM-слоя.

Применение и ограничения BiLSTM

В классической RNN/LSTM информация течет только из прошлого в будущее. Это означает, что при анализе слова на позиции *t* сеть «знает» только то, что было до него. Однако во многих задачах понимание зависит от полного окружения.

Пример: В предложении *«Я видел **bank** у реки»* слово *bank* означает «берег», а в *«Я ограбил **bank**»* - «банк». Однонаправленная сеть на слове *bank* еще не видела слова *«реки»* и может ошибиться. BiLSTM, благодаря обратному проходу, уже «заглянула» в конец предложения и использует слово *«реки»* для правильной интерпретации.

Ограничения:

- **невозможность онлайн-обработки** - поскольку обратному слою нужна вся последовательность для начала работы, BiLSTM не может делать предсказания «на лету» по мере поступления данных;
- **удвоение параметров** - количество весов увеличивается вдвое по сравнению с однонаправленной LSTM.

BiLSTM является стандартом для задач **разметки последовательностей**, где важен глобальный контекст:

- **NER** (распознавание именованных сущностей) - в составе архитектуры BiLSTM-CRF;
- **POS-теггинг** (определение частей речи);
- **Машинный перевод** - в энкодере модели Seq2Seq;
- **Анализ тональности** текста.

Глубокие (Stacked) LSTM (2013) - 1

Алекс Грейвс (Alex Graves), Абдель-Рахман Мохамед (Abdel-rahman Mohamed), Джеффри Хинтон (Geoffrey Hinton) в 2013 г. в статье «Speech recognition with deep recurrent neural networks» («Распознавание речи с использованием глубоких рекуррентных нейронных сетей») предложили глубокие нейронные сети.

Подобно тому, как глубокие сети прямого распространения изучают иерархические признаки (от простых граней к сложным объектам), многослойные рекуррентные сети (Stacked RNN/LSTM) способны моделировать временные зависимости на **разных уровнях абстракции**. Выход скрытого состояния одного LSTM-слоя служит входом для следующего LSTM-слоя, что позволяет сети строить более сложные представления последовательности.

Для *N* слоёв (*l*=1,…,*N*) скрытое состояние слоя *l* в момент времени *t* **вычисляется рекурсивно:** $h_t^{(l)} = \mathrm{LSTM}_l\left(h_t^{(l-1)}, h_{t-1}^{(l)}, C_{t-1}^{(l)}\right)$

- $h_t^{(0)} = x_t$ — входная последовательность,
- $h_t^{(l-1)}$ — выход предыдущего слоя (вход для текущего),
- $h_{t-1}^{(l)}$ и $C_{t-1}^{(l)}$ — предыдущие состояния текущего слоя.

Или по формуле с остаточной связью:

$$h_t^{(l)} = \mathrm{LSTM}_l(h_t^{(l-1)}, h_{t-1}^{(l)}, c_{t-1}^{(l)}) + h_t^{(l-1)}$$

Каждый слой имеет собственные наборы параметров (матрицы и смещения). Выход всей глубокой сети может быть снят либо с последнего слоя $h_t^{(N)}$, либо с нескольких слоёв одновременно.

Глубокие (Stacked) LSTM (2013) - 2

В работе Грейвса и др. на задаче распознавания фонем (датасет TIMIT) было показано, что:

•ошибка классификации фонем **монотонно убывает с ростом глубины** от 1 до 5 слоёв LSTM;

•глубокая LSTM с 3+ слоями достигла ошибки 19.6% и ниже;

•лучшая модель (глубинная двунаправленная LSTM) показала **рекордную на тот момент ошибку 17.7%** , превзойдя предыдущий рекорд 20.7%, установленный глубокими сетями прямого распространения.

Практические рекомендации для архитектуры глубоких LSTM:

•рекомендуемая глубина 2-4 слоя;

•батч-нормализация: применяется между слоями для стабилизации обучения;

•использование регуляризации: для глубоких LSTM критически важны dropout (на нерекуррентные связи) и gradient clipping;

•использование остаточных связей (Residual): при глубине ≥3 слоёв они помогают бороться с затуханием градиента между слоями.

Слой	Уровень абстракции	Примеры в NLP
Слой 1	Локальные паттерны	Морфемы, символьные N-граммы
Слой 2	Составные структуры	Части речи, именные группы
Слой 3+	Глобальный контекст	Синтаксис, кореференция

Кореференция - это отношение тождества референтов: разные языковые выражения в тексте указывают на один и тот же объект реального мира или понятие (Мария - она).

Эхо-сети (2001) - 1

Эхо-сети (Echo State Networks, ESN) предложены Гербертом Йегером (Herbert Jaeger) в 2001 г. в работе «The 'echo state' approach to analysing and training recurrent neural networks» («Подход 'эхо-состояния' к анализу и обучению рекуррентных нейронных сетей») и представляют собой подход к рекуррентным вычислениям, принципиально отличный от градиентных методов (BPTT).

Вместо обучения рекуррентных связей, ESN использует **фиксированный случайный резервуар** (reservoir) - большой скрытый слой с фиксированными, случайно инициализированными весами. Обучается только выходной слой, который отображает состояния резервуара в целевые значения.

1. Резервуар (не обучается): $h_t = \sigma\left(W_{in}x_t + W_{res}h_{t-1}\right)$,

- W_{in} — матрица входных весов (случайная, фиксированная),
- W_{res} — матрица весов резервуара (случайная, разреженная, фиксированная),
- σ — функция активации (обычно $\tanh$).

2. Выходной слой (обучается): $y_t = W_{out}h_t$,

где W_{out} - обучаемая матрица выходных весов, обычно методом линейной регрессии.

Для успешной работы ESN резервуар должен обладать **свойством эха**: влияние начального состояния h_0 должно экспоненциально затухать со временем и состояние резервуара h_t должно однозначно определяться историей входных сигналов $x_1,\ldots,x_t$. Это гарантируется, если **спектральный радиус** матрицы W_{res} меньше единицы.

Эхо-сети (2001) - 2

Преимущества ESN:

1. Очень быстрое обучение: только линейная регрессия для W_{out}.
2. Не страдает от затухания/взрыва градиента (рекуррентные связи не обучаются).
3. Детерминированное обучение: нет локальных минимумов.
4. Хорошо работает на малых датасетах: меньше параметров для обучения.

Недостатки ESN:

1. Требует большого резервуара (тысячи нейронов) для сложных задач.
2. Чувствителен к гиперпараметрам: спектральный радиус матрицы W_{res}, разреженность, масштабирование входа.
3. Качество обычно ниже, чем у хорошо настроенной LSTM.
4. Плохо масштабируется на сложные задачи.

Идеи ESN получили развитие в более широкой парадигме **Reservoir Computing** (вычисления в резервуаре, RC). В 2002 г. Маасс и Маркрам предложили схожую концепцию **Liquid State Machines** (машины жидкого состояния, LSM) для импульсных нейронных сетей. Обзор Лукозявичуса и Йегера в 2009 г. систематизировал различные подходы к резервуарным вычислениям.

Квазzи-рекуррентные нейронные сети (2017)

Предложены Джеймсом Брэдбери, Стивеном Мерити, Каймином Сюном, Ричардом Сочером (James Bradbury, Stephen Merity, Caiming Xiong, Richard Socher) в работе «Quasi-Recurrent Neural Networks» в 2017 г.

Квазzи-рекуррентные нейронные сети (Quasi-Recurrent Neural Networks, QRNN) - это гибридная архитектура, которая стремится объединить сильные стороны свёрточных сетей (CNN) и рекуррентных сетей (RNN). Её ключевая цель - преодолеть главный недостаток LSTM и GRU: **невозможность распараллеливания по временной оси** из-за зависимости каждого шага от предыдущего.

В отличие от LSTM, где вычисления на шаге t строго зависят от h_{t-1}, QRNN разделяет обработку последовательности на два этапа:

1. **Свёрточный компонент (Convolutional Component):** выполняет параллельные вычисления над всеми временными шагами одновременно, как в CNN.
2. **Рециркуляционный пулинг (Recurrent Pooling):** выполняет минимальную рекуррентную обработку, которая действует независимо по каналам признаков.

Преимущества QRNN:

•высокая скорость - особенно эффективны на длинных последовательностях и малых батчах;

•хорошее масштабирование - подходят для символьных задач, где длина последовательности может достигать тысяч;

•может использоваться как замена LSTM/GRU в существующих архитектурах.

Простой рекуррентный блок (2017)

Предложены Тао Лэй, Ю Чжан, Йоав Арци (Tao Lei, Yu Zhang, Yoav Artzi) в работе «Training RNNs as Fast as CNNs» (Обучение RNN так же быстро, как CNN) в 2017 г.

Простой рекуррентный блок (Simple Recurrent Unit, SRU) - это упрощённая рекуррентная архитектура, которая идёт ещё дальше QRNN в стремлении максимизировать параллелизацию.

Её ключевая инновация - **полное отделение рекуррентной зависимости от матричных умножений**. В SRU вентили вычисляются только на основе текущего входа x_t без зависимости от h_{t-1}, а все матричные умножения вынесены за пределы рекуррентного цикла. Добавлено шлюзовое соединение **Highway Connection**, по которому часть входного сигнала проходит напрямую в выход, минуя нелинейные преобразования.

Преимущества SRU:

- максимальная параллелизация - все матричные умножения вынесены из рекурсии, что позволяет выполнить их за один проход для всей последовательности;
- высокая скорость на CPU - в экспериментах на ARM CPU SRU показал ускорение до 930% при обработке 16 временных шагов за раз;
- устойчивость глубоких сетей: Highway Connection облегчает обучение многослойных архитектур;
- простота реализации - меньше операций, чем в LSTM, легче оптимизировать под конкретное железо.

Сравнение рекуррентных архитектур

Архитектура	Параметры	Параллелизация	Длинные зависимости	Скорость обучения	Год	Статус
Simple RNN	1×	Нет	Плохо (≤20)	Высокая	1990	Устарела
LSTM	4×	Нет	**Отлично** (сотни)	Средняя	1997	**Стандарт**
GRU	3×	Нет	Хорошо	Выше средней	2014	**Стандарт**
Peephole LSTM	4× +	Нет	Отлично	Средняя	2000	Нишевое
BiLSTM	8×	Нет	Отлично (оба напр.)	Низкая	2005	**Стандарт для NLP**
Stacked LSTM	4L×	Нет	Отлично	Низкая	2013	Глубокие задачи
ESN	1× (только выход)	Нет	Хорошо	Очень высокая	2001	Малые датасеты
QRNN	~2×	**Да**	Хорошо	Высокая	2017	Альтернатива
SRU	~2×	**Да**	Хорошо	Очень высокая	2017	Альтернатива

Рекомендации по выбору архитектуры

Сценарий	Рекомендуемая архитектура	Обоснование
Короткие последовательности (<100 шагов), важна максимальная точность	**LSTM** или **BiLSTM**	Проверенный стандарт, отличная работа с долгосрочными зависимостями, богатая экосистема предобученных моделей.
Средние последовательности (100-500 шагов), ограниченные данные	**GRU**	Меньше параметров → меньше риск переобучения, быстрее обучается, качество сопоставимо с LSTM.
Длинные последовательности (>500 шагов), важна скорость обучения	**QRNN**	Параллельная свёрточная часть даёт огромный выигрыш в скорости на GPU, качество близко к LSTM.
Инференс на CPU или мобильных устройствах	**SRU**	Минимум операций, highway-соединения для стабильности, отличная оптимизация под ARM-процессоры.
Потоковая обработка в реальном времени	**GRU** или **SRU**	Однонаправленные, константная память $O(1)$, малая задержка на шаг.
Задачи с двусторонним контекстом (NER, разметка)	**BiLSTM** или **BiGRU**	Доступ к полному контексту критически важен, офлайн-обработка допустима.
Малые датасеты, быстрая адаптация к новым данным	**ESN**	Обучение только выходного слоя линейной регрессией, нет проблем с градиентами, очень быстро.

Текущие тренды и будущее

Несмотря на доминирование трансформеров в NLP, рекуррентные архитектуры продолжают активно развиваться в трёх направлениях:

1. **Гибридизация с вниманием:** Современные модели всё чаще комбинируют RNN-компоненты с механизмами внимания, заимствуя сильные стороны обоих подходов (например, Tacotron 2, где BiLSTM-энкодер сочетается с Location-Sensitive Attention).
2. **Сверхдлинные последовательности:** State Space Models (S4, Mamba) и RWKV наследуют идеи эффективной рекуррентности, достигая линейной сложности и константной памяти при инференсе, что открывает возможности для обработки последовательностей длиной в миллионы токенов.
3. **Аппаратно-ориентированный дизайн:** Новые архитектуры (QRNN, SRU) проектируются с учётом особенностей современных ускорителей (GPU, TPU, NPU), максимизируя степень параллелизма и минимизируя операции, неэффективные на конкретном железе.

Пример работы LSTM - 1

Предположим, мы обрабатываем предложение: «Мария живёт в Париже. Она работает дизайнером.»

Когда сеть доходит до слова «Она», вентиль забывания может решить, что информация о том, что «Мария» - женщина, важна для правильного согласования, и сохранит её ($f_t \approx 1$).

А информация о том, что Мария «живёт», уже не так важна для текущего момента, и её можно частично забыть ($ft \approx 0.3$).

На слове «работает» входной вентиль может решить, что профессия «дизайнер» - это важная новая информация, которую стоит добавить в память.

Вентиль i_t откроется (значения близки к 1) для тех элементов памяти, которые отвечают за профессию, а кандидат $C\tilde{}_t$ предложит значения, кодирующие информацию о дизайнере.

Когда сеть генерирует слово «дизайнером», выходной гейт должен извлечь из памяти информацию о профессии, добавленную на слове «работает», и подавить нерелевантную информацию (например, о месте жительства).

Пример работы LSTM - 2

Рассмотрим **пример** прямого хода LSTM для текста «Мария живёт в Париже. Она работает дизайнером.»

Нотация весов:

Группа	Матрица для входа x_t	Матрица для скрытого состояния h_{t-1}	Смещение b
Forget Gate	W_{fx}	W_{fh}	b_f
Input Gate	W_{ix}	W_{ih}	b_i
Candidate	W_{Cx}	W_{Ch}	b_C
Output Gate	W_{ox}	W_{oh}	b_o

Размерности для примера:

- Входной вектор $x_t \in \mathbb{R}^4$ (мы расширим до 4 признаков)
- Скрытый вектор $h_t \in \mathbb{R}^3$ (3 нейрона памяти: Род, Действие, Место/Профессия)
- $W_{*x} \in \mathbb{R}^{3\times4}$
- $W_{*h} \in \mathbb{R}^{3\times3}$

Пример работы LSTM - 3

Эмбеддинг (вложение) - это числовое представление какого-либо объекта в виде вектора фиксированной длины (набора чисел), которое отражает его ключевые свойства и взаимосвязи с другими объектами. То сеть, эмбеддинг переводит сложные, разнородные данные (слова, изображения, пользователей и т. д.) в понятный для компьютера числовой формат - вектор в многомерном пространстве.

Пусть размерность эмбеддинга d=4. Каждое измерение отвечает за конкретный семантический признак:

1. **Dim 1:** *Одушевлённость / Человек* (1 = человек, 0 = предмет, -1 = место)
2. **Dim 2:** *Женский род* (1 = женский, 0 = мужской/средний)
3. **Dim 3:** *Глагольность / Действие* (1 = действие, 0 = состояние/объект)
4. **Dim 4:** *Профессия / Род деятельности* (1 = профессия, 0 = нет)

Пример работы LSTM - 4

Назначим числа каждому слову из текста:

Слово	Вектор x	Логика назначения
Мария	`[0.9, 1.0, 0.0, 0.0]`	Человек (0.9), Женщина (1.0), Не глагол (0.0), Не профессия (0.0)
живёт	`[0.5, 0.0, 0.8, 0.0]`	Одуш. субъект (0.5), Глагол состояния (0.8)
в	`[0.0, 0.0, 0.0, 0.0]`	Служебное слово
Париже	`[-0.8, 0.0, 0.1, 0.0]`	Место (-0.8), Статика (0.1)
Она	`[1.0, 1.0, 0.0, 0.0]`	Почти идентичен "Марии"
работает	`[0.5, 0.0, 1.0, 0.3]`	Глагол действия (1.0), Намёк на профессию (0.3)
дизайнером	`[0.8, 0.0, 0.0, 1.0]`	Человек (0.8), Явная профессия (1.0)

Матрица эмбеддингов $E \in \mathbb{R}^{7\times4}$:

$$E = \begin{pmatrix} 0.9 & 1.0 & 0.0 & 0.0 \\ 0.5 & 0.0 & 0.8 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 \\ -0.8 & 0.0 & 0.1 & 0.0 \\ 1.0 & 1.0 & 0.0 & 0.0 \\ 0.5 & 0.0 & 1.0 & 0.3 \\ 0.8 & 0.0 & 0.0 & 1.0 \end{pmatrix}$$

Пример работы LSTM - 5

Мы хотим, чтобы сеть:

1. Сохраняла род при появлении местоимения «Она» (Forget Gate ≈ 1).
2. Забывала глагол «живёт» (Forget Gate ≈ 0.3).
3. Записывала профессию при появлении слов «работает» и «дизайнером» (Input Gate ≈ 1).
4. Извлекала профессию на выходе (Output Gate).

Нейроны памяти C_t:

- **Нейрон 1:** хранит признак **Женский род**.
- **Нейрон 2:** хранит признак **Действие / Глагол**.
- **Нейрон 3:** хранит признак **Место / Профессия** (в начале место, потом перезаписывается профессией).

Для этого специально **подберем** матрицы параметров и сдвиги. Пока без обучения, просто для демонстрации работы LSTM.

Инициализация матриц весов

Группа Forget Gate (W_{fx}, W_{fh}, b_f):

$$W_{fx} = \begin{pmatrix} 0 & 2 & 0 & 0 \\ 0 & 0 & -2 & 0 \\ -1 & 0 & 0 & 0 \end{pmatrix} \quad W_{fh} = \begin{pmatrix} 0.1 & 0 & 0 \\ 0 & 0.1 & 0 \\ 0 & 0 & 0.1 \end{pmatrix} \quad b_f = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

Строка 1 даёт высокий сигнал при наличии женского рода, сигмоида будет близка к 1 (сохраняем). Строка 2 даёт отрицательный сигнал при виде нового глагола, сигмоида будет около 0.3 (забываем старый глагол).

Группа Input Gate (W_{ix},W_{ih},b_i):

$$W_{ix} = \begin{pmatrix} 0 & 2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 3 \end{pmatrix} \quad W_{ih} = \begin{pmatrix} 0.1 & 0 & 0 \\ 0 & 0.1 & 0 \\ 0 & 0 & 0.1 \end{pmatrix} \quad b_i = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

Строка 3 даёт большой сигнал, когда видит слово с признаком профессии (например, «дизайнером» с x_4=1.0 даст 3.0, сигмоида ≈ 0.95).

Группа Candidate (W_{Cx}, W_{Ch}, b_C): $W_{Cx} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$ $W_{Ch} = 0, b_C = 0$

Группа Output Gate (W_{ox}, W_{oh}, b_o): $W_{ox} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix}$

$W_{oh} \in \mathbb{R}^{3\times 3}$: диагональная с 0.1

$b_o = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$

Начальные состояния: $h_0 = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}, C_0 = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$

Прямой ход - 1

Шаг 1: "Мария" ($x_1 = [0.9, 1.0, 0.0, 0.0]^T$)

1.1. Линейные преобразования:

$$z_f = W_{fx}x_1 = \begin{pmatrix} 0 & 2 & 0 & 0 \\ 0 & 0 & -2 & 0 \\ -1 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} 0.9 \\ 1.0 \\ 0.0 \\ 0.0 \end{pmatrix} = \begin{pmatrix} 2.0 \\ 0.0 \\ -0.9 \end{pmatrix}$$

$$z_i = W_{ix}x_1 = \begin{pmatrix} 0 & 2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 3 \end{pmatrix} \begin{pmatrix} 0.9 \\ 1.0 \\ 0.0 \\ 0.0 \end{pmatrix} = \begin{pmatrix} 2.0 \\ 0.0 \\ 0.0 \end{pmatrix}$$

$$z_C = W_{Cx}x_1 = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0.9 \\ 1.0 \\ 0.0 \\ 0.0 \end{pmatrix} = \begin{pmatrix} 1.0 \\ 0.0 \\ 0.0 \end{pmatrix}$$

$$z_o = W_{ox}x_1 = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix} \begin{pmatrix} 0.9 \\ 1.0 \\ 0.0 \\ 0.0 \end{pmatrix} = \begin{pmatrix} 1.0 \\ 0.0 \\ 0.0 \end{pmatrix}$$

Прямой ход - 2

1.2. Активации:

$$f_1 = \sigma\left(\begin{pmatrix} 2.0 \\ 0.0 \\ -0.9 \end{pmatrix}\right) \approx \begin{pmatrix} 0.88 \\ 0.50 \\ 0.29 \end{pmatrix}$$

$$i_1 = \sigma\left(\begin{pmatrix} 2.0 \\ 0.0 \\ 0.0 \end{pmatrix}\right) \approx \begin{pmatrix} 0.88 \\ 0.50 \\ 0.50 \end{pmatrix}$$

$$\tilde{C}_1 = \tanh\left(\begin{pmatrix} 1.0 \\ 0.0 \\ 0.0 \end{pmatrix}\right) \approx \begin{pmatrix} 0.76 \\ 0.0 \\ 0.0 \end{pmatrix}$$

$$o_1 = \sigma\left(\begin{pmatrix} 1.0 \\ 0.0 \\ 0.0 \end{pmatrix}\right) \approx \begin{pmatrix} 0.73 \\ 0.50 \\ 0.50 \end{pmatrix}$$

Прямой ход - 3

1.2. Активации:

$$f_1 = \sigma\left(\begin{pmatrix}2.0\\0.0\\-0.9\end{pmatrix}\right) \approx \begin{pmatrix}0.88\\0.50\\0.29\end{pmatrix}$$

$$i_1 = \sigma\left(\begin{pmatrix}2.0\\0.0\\0.0\end{pmatrix}\right) \approx \begin{pmatrix}0.88\\0.50\\0.50\end{pmatrix}$$

$$\tilde{C}_1 = \tanh\left(\begin{pmatrix}1.0\\0.0\\0.0\end{pmatrix}\right) \approx \begin{pmatrix}0.76\\0.0\\0.0\end{pmatrix}$$

$$o_1 = \sigma\left(\begin{pmatrix}1.0\\0.0\\0.0\end{pmatrix}\right) \approx \begin{pmatrix}0.73\\0.50\\0.50\end{pmatrix}$$

1.3. Состояния:

$$C_1 = 0 + i_1 \odot \tilde{C}_1 = \begin{pmatrix}0.88 \times 0.76\\0\\0\end{pmatrix} = \begin{pmatrix}0.67\\0\\0\end{pmatrix}$$

В памяти сохраняется: Нейрон 1 (Род) = 0.67, Нейрон 2 (Действие) = 0, Нейрон 3 (Место/Профессия) = 0.

Прямой ход - 4

Шаг 2: "живёт" ($x_2 = [0.5, 0.0, 0.8, 0.0]^T$)

2.1. Линейные преобразования:

$$z_f = W_{fx}x_2 + W_{fh}h_1 \approx \begin{pmatrix} 0 & 2 & 0 & 0 \\ 0 & 0 & -2 & 0 \\ -1 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} 0.5 \\ 0.0 \\ 0.8 \\ 0.0 \end{pmatrix} = \begin{pmatrix} 0.0 \\ -1.6 \\ -0.5 \end{pmatrix}$$

$$z_i = W_{ix}x_2 = \begin{pmatrix} 0 & 2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 3 \end{pmatrix} \begin{pmatrix} 0.5 \\ 0.0 \\ 0.8 \\ 0.0 \end{pmatrix} = \begin{pmatrix} 0.0 \\ 0.8 \\ 0.0 \end{pmatrix}$$

$$z_C = W_{Cx}x_2 = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0.5 \\ 0.0 \\ 0.8 \\ 0.0 \end{pmatrix} = \begin{pmatrix} 0.0 \\ 0.8 \\ 0.0 \end{pmatrix}$$

Прямой ход - 5

2.2. Активации:

$$f_2 = \sigma\left(\begin{pmatrix}0.0\\-1.6\\-0.5\end{pmatrix}\right) \approx \begin{pmatrix}0.50\\0.17\\0.38\end{pmatrix}$$

f2[2]≈0.17, следовательно, Forget Gate для Нейрона 2 (Действие) почти закрыт. Это значит, что предыдущее действие (а его не было) будет забыто, но если бы было, то бы стёрлось.

$$i_2 = \sigma\left(\begin{pmatrix}0.0\\0.8\\0.0\end{pmatrix}\right) \approx \begin{pmatrix}0.50\\0.69\\0.50\end{pmatrix}$$

$$\tilde{C}_2 = \tanh\left(\begin{pmatrix}0.0\\0.8\\0.0\end{pmatrix}\right) \approx \begin{pmatrix}0.0\\0.66\\0.0\end{pmatrix}$$

2.3. Состояния:

$$C_2 = f_2 \odot C_1 + i_2 \odot \tilde{C}_2$$

$$C_2 = \begin{pmatrix}0.50 \times 0.67\\0.17 \times 0\\0.38 \times 0\end{pmatrix} + \begin{pmatrix}0\\0.69 \times 0.66\\0\end{pmatrix} = \begin{pmatrix}0.34\\0.46\\0\end{pmatrix}$$

В памяти сохраняется: Род = 0.34, Действие («живёт») = 0.46, Место = 0

Шаг 3: "в" ($x_3 \approx 0$) — пропускаем, память почти не меняется.

Прямой ход - 6

Шаг 4: "Париже" ($x_4 = [-0.8, 0.0, 0.1, 0.0]^T$)

4.1. Линейные преобразования:

$$z_f = W_{fx}x_4 = \begin{pmatrix} 0 & 2 & 0 & 0 \\ 0 & 0 & -2 & 0 \\ -1 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} -0.8 \\ 0.0 \\ 0.1 \\ 0.0 \end{pmatrix} = \begin{pmatrix} 0.0 \\ -0.2 \\ 0.8 \end{pmatrix}$$

$$z_i = W_{ix}x_4 = \begin{pmatrix} 0 & 2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 3 \end{pmatrix} \begin{pmatrix} -0.8 \\ 0.0 \\ 0.1 \\ 0.0 \end{pmatrix} = \begin{pmatrix} 0.0 \\ 0.1 \\ 0.0 \end{pmatrix}$$

$$z_C = W_{Cx}x_4 = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} -0.8 \\ 0.0 \\ 0.1 \\ 0.0 \end{pmatrix} = \begin{pmatrix} 0.0 \\ 0.1 \\ 0.0 \end{pmatrix}$$

Прямой ход - 7

4.2. Активации:

$$f_4 = \sigma\left(\begin{pmatrix}0.0\\-0.2\\0.8\end{pmatrix}\right) \approx \begin{pmatrix}0.50\\0.45\\0.69\end{pmatrix}$$

$$i_4 = \sigma\left(\begin{pmatrix}0.0\\0.1\\0.0\end{pmatrix}\right) \approx \begin{pmatrix}0.50\\0.52\\0.50\end{pmatrix}$$

$$\tilde{C}_4 = \tanh\left(\begin{pmatrix}0.0\\0.1\\0.0\end{pmatrix}\right) \approx \begin{pmatrix}0.0\\0.1\\0.0\end{pmatrix}$$

4.3. Состояния:

$$C_4 = f_4 \odot C_3 + i_4 \odot \tilde{C}_4 \approx \begin{pmatrix}0.50 \times 0.34\\0.45 \times 0.46\\0.69 \times 0\end{pmatrix} + 0 = \begin{pmatrix}0.17\\0.21\\0\end{pmatrix}$$

В памяти сохраняется: Род = 0.17 (ослаб, но жив), Действие = 0.21 (затухает).

Прямой ход - 8

Шаг 5: "Она" ($x_5 = [1.0, 1.0, 0.0, 0.0]^T$) — МОМЕНТ ИСТИНЫ

5.1. Forget Gate:

$$z_f = W_{fx} x_5 = \begin{pmatrix} 0 & 2 & 0 & 0 \\ 0 & 0 & -2 & 0 \\ -1 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} 1.0 \\ 1.0 \\ 0.0 \\ 0.0 \end{pmatrix} = \begin{pmatrix} 2.0 \\ 0.0 \\ -1.0 \end{pmatrix}$$

$$f_5 = \sigma\left(\begin{pmatrix} 2.0 \\ 0.0 \\ -1.0 \end{pmatrix}\right) \approx \begin{pmatrix} \mathbf{0.88} \\ \mathbf{0.50} \\ 0.27 \end{pmatrix}$$

- **Нейрон 1 (Род):** $f_5[1] \approx 0.88$ — **СОХРАНЯЕМ** (почти 1.0, как и требовалось в сценарии).
- **Нейрон 2 (Действие):** $f_5[2] = 0.50$ — нейтрально.
- **Нейрон 3 (Место):** $f_5[3] = 0.27$ — **ЗАБЫВАЕМ** место.

Прямой ход - 9

5.2. Input Gate:

$$z_i = W_{ix}x_5 = \begin{pmatrix} 0 & 2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 3 \end{pmatrix} \begin{pmatrix} 1.0 \\ 1.0 \\ 0.0 \\ 0.0 \end{pmatrix} = \begin{pmatrix} 2.0 \\ 0.0 \\ 0.0 \end{pmatrix}$$

$$i_5 = \sigma(\begin{pmatrix} 2.0 \\ 0.0 \\ 0.0 \end{pmatrix}) \approx \begin{pmatrix} 0.88 \\ 0.50 \\ 0.50 \end{pmatrix}$$

$$\tilde{C}_5 = \tanh(W_{Cx}x_5) = \tanh(\begin{pmatrix} 1.0 \\ 0.0 \\ 0.0 \end{pmatrix}) \approx \begin{pmatrix} 0.76 \\ 0.0 \\ 0.0 \end{pmatrix}$$

5.3. Обновление C_5:

$$C_5 = f_5 \odot C_4 + i_5 \odot \tilde{C}_5$$

$$C_5 = \begin{pmatrix} 0.88 \times 0.17 \\ 0.50 \times 0.21 \\ 0.27 \times 0.0 \end{pmatrix} + \begin{pmatrix} 0.88 \times 0.76 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0.15 \\ 0.11 \\ 0.0 \end{pmatrix} + \begin{pmatrix} 0.67 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} \mathbf{0.82} \\ 0.11 \\ 0.0 \end{pmatrix}$$

Результат: Нейрон 1 (Род) восстановлен до 0.82. Нейрон 2 (Действие «живёт») затух до 0.11 (частично забыт, как и требовалось). Нейрон 3 (Место) обнулён.

Прямой ход - 10

Шаг 6: "работает" ($x_6 = [0.5, 0.0, 1.0, 0.3]^T$)

6.1. Входной вентиль (Input Gate) — запись профессии:

$$z_i = W_{ix} x_6 = \begin{pmatrix} 0 & 2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 3 \end{pmatrix} \begin{pmatrix} 0.5 \\ 0.0 \\ 1.0 \\ 0.3 \end{pmatrix} = \begin{pmatrix} 0.0 \\ 1.0 \\ 0.9 \end{pmatrix}$$

$$i_6 = \sigma\left(\begin{pmatrix} 0.0 \\ 1.0 \\ 0.9 \end{pmatrix} \right) \approx \begin{pmatrix} 0.50 \\ 0.73 \\ \mathbf{0.71} \end{pmatrix}$$

Input Gate для Нейрона 3 (Профессия) открылся на 0.71.

Прямой ход - 11

6.2. Кандидат:

$$\tilde{C}_6 = \tanh(W_{Cx}x_6) = \tanh\left(\begin{pmatrix}0.0\\1.0\\0.3\end{pmatrix}\right) \approx \begin{pmatrix}0.0\\0.76\\0.29\end{pmatrix}$$

6.3. Forget Gate:

$$z_f = W_{fx}x_6 = \begin{pmatrix}0 & 2 & 0 & 0\\0 & 0 & -2 & 0\\-1 & 0 & 0 & 0\end{pmatrix}\begin{pmatrix}0.5\\0.0\\1.0\\0.3\end{pmatrix} = \begin{pmatrix}0.0\\-2.0\\-0.5\end{pmatrix}$$

$$f_6 = \sigma\left(\begin{pmatrix}0.0\\-2.0\\-0.5\end{pmatrix}\right) \approx \begin{pmatrix}0.50\\0.12\\0.38\end{pmatrix}$$

6.4. Обновление C_6:

$$C_6 = f_6 \odot C_5 + i_6 \odot \tilde{C}_6$$

$$C_6 = \begin{pmatrix}0.50 \times 0.82\\0.12 \times 0.11\\0.38 \times 0.0\end{pmatrix} + \begin{pmatrix}0\\0.73 \times 0.76\\0.71 \times 0.29\end{pmatrix} = \begin{pmatrix}0.41\\0.01\\0.0\end{pmatrix} + \begin{pmatrix}0\\0.55\\0.21\end{pmatrix} = \begin{pmatrix}0.41\\0.56\\\mathbf{0.21}\end{pmatrix}$$

В памяти сохраняется: Род = 0.41, Действие «работает» = 0.56, Профессия = 0.21.

Прямой ход - 12

Шаг 7: "дизайнером" ($x_7 = [0.8, 0.0, 0.0, 1.0]^T$)

7.1. Входной вентиль — МАКСИМАЛЬНОЕ ОТКРЫТИЕ:

$$z_i = W_{ix} x_7 = \begin{pmatrix} 0 & 2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 3 \end{pmatrix} \begin{pmatrix} 0.8 \\ 0.0 \\ 0.0 \\ 1.0 \end{pmatrix} = \begin{pmatrix} 0.0 \\ 0.0 \\ 3.0 \end{pmatrix}$$

$$i_7 = \sigma\left(\begin{pmatrix} 0.0 \\ 0.0 \\ 3.0 \end{pmatrix} \right) \approx \begin{pmatrix} 0.50 \\ 0.50 \\ \mathbf{0.95} \end{pmatrix}$$

Input Gate для Нейрона 3 открыт на 0.95.

Прямой ход - 13

7.2. Кандидат:

$$\tilde{C}_7 = \tanh(W_{Cx}x_7) = \tanh(\begin{pmatrix}0.0\\0.0\\1.0\end{pmatrix}) \approx \begin{pmatrix}0.0\\0.0\\0.76\end{pmatrix}$$

7.3. Forget Gate:

$$z_f = W_{fx}x_7 = \begin{pmatrix}0 & 2 & 0 & 0\\0 & 0 & -2 & 0\\-1 & 0 & 0 & 0\end{pmatrix}\begin{pmatrix}0.8\\0.0\\0.0\\1.0\end{pmatrix} = \begin{pmatrix}0.0\\0.0\\-0.8\end{pmatrix}$$

$$f_7 = \sigma(\begin{pmatrix}0.0\\0.0\\-0.8\end{pmatrix}) \approx \begin{pmatrix}0.50\\0.50\\0.31\end{pmatrix}$$

7.4. Обновление C_7:

$$C_7 = f_7 \odot C_6 + i_7 \odot \tilde{C}_7$$

$$C_7 = \begin{pmatrix}0.50 \times 0.41\\0.50 \times 0.56\\0.31 \times 0.21\end{pmatrix} + \begin{pmatrix}0\\0\\0.95 \times 0.76\end{pmatrix} = \begin{pmatrix}0.21\\0.28\\0.07\end{pmatrix} + \begin{pmatrix}0\\0\\0.72\end{pmatrix} = \begin{pmatrix}0.21\\0.28\\\mathbf{0.79}\end{pmatrix}$$

В памяти сохраняется: Род = 0.21, Действие = 0.28, Профессия = 0.79 (ярко выражена).

Прямой ход - 14

7.5. Выходной гейт (Output Gate) — извлечение профессии:

$$z_o = W_{ox} x_7 = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix} \begin{pmatrix} 0.8 \\ 0.0 \\ 0.0 \\ 1.0 \end{pmatrix} = \begin{pmatrix} 0.0 \\ 0.0 \\ 2.0 \end{pmatrix}$$

$$o_7 = \sigma\left(\begin{pmatrix} 0.0 \\ 0.0 \\ 2.0 \end{pmatrix} \right) \approx \begin{pmatrix} 0.50 \\ 0.50 \\ \mathbf{0.88} \end{pmatrix}$$

Output Gate широко открыт для Нейрона 3 (Профессия).

7.6. Скрытое состояние h_7 (выход сети):

$$h_7 = o_7 \odot \tanh(C_7) = \begin{pmatrix} 0.50 \\ 0.50 \\ 0.88 \end{pmatrix} \odot \tanh\left(\begin{pmatrix} 0.21 \\ 0.28 \\ 0.79 \end{pmatrix} \right)$$

$$\tanh(C_7) \approx \begin{pmatrix} 0.21 \\ 0.27 \\ 0.66 \end{pmatrix}$$

$$h_7 = \begin{pmatrix} 0.50 \times 0.21 \\ 0.50 \times 0.27 \\ 0.88 \times 0.66 \end{pmatrix} = \begin{pmatrix} 0.11 \\ 0.14 \\ \mathbf{0.58} \end{pmatrix}$$

На выходе сети доминирует Нейрон 3 (0.58), несущий информацию о профессии «дизайнер». Информация о месте жительства (Париж) полностью подавлена (0.0 в памяти).

Итоговая таблица состояний памяти Ct

Шаг	Слово	Нейрон 1 (Род)	Нейрон 2 (Действие)	Нейрон 3 (Место/Профессия)
1	Мария	**0.67**	0.0	0.0
2	живёт	0.34	**0.46**	0.0
4	Париже	0.17	0.21	0.0
5	**Она**	**0.82**	0.11	0.0
6	работает	0.41	**0.56**	**0.21**
7	**дизайнером**	0.21	0.28	**0.79**

1. **Forget Gate на слове "Она":**
   - Для **Нейрона 1 (Род)**: $f_5[1] \approx 0.88$ (сохранили, близко к 1).
   - Для **Нейрона 2 (Действие "живёт")**: память уменьшилась с 0.21 до 0.11, то есть **частично забыта** (коэффициент 0.50).
   - Для **Нейрона 3 (Место)**: $f_5[3] = 0.27$ — забыли.
2. **Input Gate на словах "работает" и "дизайнером":**
   - На "работает" $i_6[3] = 0.71$ — начали записывать профессию.
   - На "дизайнером" $i_7[3] = 0.95$ — **максимально открылись** и записали кандидата 0.76. Профессия заняла доминирующее положение в памяти.
3. **Output Gate на слове "дизайнером":**
   - $o_7[3] = 0.88$ — сеть извлекла именно **профессию**, подавив нерелевантную информацию о роде (0.11) и действии (0.14) на выходе h_7.